生成 AI と RAG を用いた観光プラン生成アプリの実装と考察

Tavily・Gemini を用いた Ehime Tour Planner の開発

2026 年 4 月 20 日

(1) やったこと・動かしたものの概要

本課題では，生成 AI と検索拡張生成（RAG）を組み合わせた観光プラン生成システム `Ehime Tour Planner` を実際に実装し，ローカル環境および公開済みアプリ上で動作確認を行った。単に既存アプリを紹介するのではなく，`Tavily` による観光情報検索，`gemini-embedding-001` による埋め込み表現，`FAISS` を用いた関連文書検索，`gemini-2.5-flash-lite` による構造化された旅程生成という一連の AI 技術を組み合わせ，入力から出力までの流れを自分で動かして確かめた。

具体的には，利用者が旅行日数，同行者，移動手段，関心テーマ，訪問エリアなどを入力すると，まず観光情報ページを検索・取得し，その内容を埋め込みベクトルに変換して関連箇所を抽出し，その後に LLM が出典 URL 付きの旅行プランを生成する仕組みを作成した。さらに，初回プランの出力後にチャットで修正指示を与え，既存プランを再構成できることも確認した。

この課題を通じて実際に動かした対象は，単体の API 呼び出しではなく，検索，埋め込み，検索結果の選別，生成，対話的な再生成を統合した小規模な生成 AI アプリケーションである。そのため，本レポートではアプリの見た目そのものよりも，どの AI 技術をどの順序で用い，どのような処理として実装したかに重点を置いて述べる。

使用技術の要点

- UI: `Streamlit`
- 検索: `Tavily Search`（必要に応じて Web 全体も追加）
- 埋め込み: `gemini-embedding-001`
- 生成: `gemini-2.5-flash-lite`
- 検索基盤: `FAISS`，利用できない場合は NumPy のコサイン類似度で代替
- 出力: JSON スキーマ準拠の旅程と Markdown ダウンロード

(2) 具体的な手続き

1. 環境設定と実行手順

まず，生成 AI アプリを実際に動作させるための環境を整えた。本システムは Python 環境上で Streamlit を用いて実行した。必要なライブラリは requirements.txt にまとめられており，streamlit，google-genai，tavily-python，faiss-cpu，beautifulsoup4 などを利用した。

実行準備としては，まず依存関係をインストールし，つぎに .streamlit/secrets.toml に GEMINI_API_KEY と TAVILY_API_KEY を設定する。app.py では，これらの値を st.secrets または環境変数から読み込み，どちらかが欠けている場合はアプリを停止する実装になっている。

実行手順の例を以下に示す。

- pip install -r requirements.txt
- .streamlit/secrets.toml を作成し，API キーを設定
- streamlit run app.py

このようにして環境構築後，ローカル実行に加えて公開済みの Streamlit アプリでも同じ処理が動くことを確認した。公開先は以下の通りである。

https://ehime-tour-planner-iicqpkkbfs9zrcjba6a9at.streamlit.app/

2. 入力条件を受け取る画面

実装の最初の段階では，利用者が旅行条件を指定するための画面を Streamlit で構成した。旅行日数，開始日，同行者，移動手段，発着地，関心テーマ，訪問エリア，子連れ配慮，1 日の詰め込み度をサイドバーから入力できるようにしている。この段階で条件を明示的に取得しておくことで，後段の検索クエリと旅程生成プロンプトに同じ制約を反映できる。

コード 1: 条件入力 UI（app.py より）

```
# --- Sidebar: 条件入力 ---
st.sidebar.header("プラン条件")
with st.sidebar:
    trip_days = st.number_input("旅行日数（日）", 1, 14, 2)
    start_date = st.date_input("開始日（任意）", value=date.today())
    party = st.text_input("同行者（例: 大人 2・小学生 1）", "大人 2")
    transport = st.selectbox("移動手段", ["公共交通", "自家用車", "レンタカー", "自転車"], index=1)

    st.divider()
    st.markdown("##### 発着地（任意）")
    start_end_options = ["指定なし", "松山空港", "JR 松山駅", "松山市駅", "松山観光港", "その他（自由記述）"]
    start_end_choice = st.selectbox("場所を選択", start_end_options, index=0)

    start_end_point = ""
    if start_end_choice == "その他（自由記述）":
        start_end_point = st.text_input("自由記述欄", placeholder="例: 今治港、自宅など")
    else:
        start_end_point = start_end_choice
    st.divider()

    interests = st.multiselect(
        "関心テーマ",
        [
            "温泉", "城・歴史", "サイクリング",
            "自然景観", "島めぐり", "グルメ",
```

```
            "アート", "祭り・イベント", "体験・アクティビティ",
        ],
        default=["温泉", "グルメ"],
    )

    area_options = [
        "指定なし", "中予（松山・道後）",
        "東予（今治・西条など）", "南予（大洲・内子・宇和島など）",
        "その他（自由記述）",
    ]
    area_choice = st.selectbox("主な訪問エリア", area_options, index=0)

    start_area = ""
    if area_choice == "その他（自由記述）":
        start_area = st.text_input("エリアを自由に入力", placeholder="例: 愛南町、鬼北町")
    else:
        start_area = area_choice

    with_kids = st.checkbox("子連れ考慮")
    pace = st.select_slider("1 日の詰め込み度", options=["ゆったり", "標準", "ぎっしり"], value="標準")
    generate_btn = st.button("プラン生成", type="primary")
```

3. 関連ページの収集と前処理

次に，RAG の入力となる観光情報を集める処理を実装した。このアプリでは Tavily を用い，既定では iyokannet.jp にドメインを限定して検索する。必要に応じて Web 全体の検索結果も加えられるようにしつつ，URL の重複は除去し，取得した本文は BeautifulSoup でテキスト化してから保持する。これにより，出典を保ちつつも，そのままの転載ではない再利用可能な検索用データを作成できる。

コード 2: 検索と前処理（rag/retriever.py より）

```
    def search_and_prepare(
        self,
        query: str,
        max_results: int = 8,
        add_web_search: bool = False,
    ) -> List[RetrievalItem]:
        items: List[RetrievalItem] = []
        seen_urls = set()

        # 配分を決定
        if add_web_search:
            iyokan_results_count = max_results // 2
            web_results_count = max_results - iyokan_results_count
        else:
            iyokan_results_count = max_results
            web_results_count = 0

        def _process_results(results, is_iyokan: bool):
            for r in results:
                url = r.get("url", "")
                if not url or url in seen_urls:
                    continue

                title = r.get("title", "") or url
                raw_md = r.get("raw_content", "") or "\n".join(r.get("content", []))

                site_name = "いよ観ネット" if is_iyokan else url.split('/')[2].replace("www.", "")

                cleaned = self._clean_text(raw_md)
                if not cleaned:
                    try:
                        ext = self.client.extract(url)
                        cleaned = self._clean_text(ext.get("text", ""))
                    except Exception:
                        continue

                if cleaned:
                    items.append(RetrievalItem(
                        title=title[:180],
                        url=url,
                        site=site_name,
                        content=cleaned[:10000],
                        content_chars=len(cleaned),
                    ))
                    seen_urls.add(url)

        # 1. いよ観ネットを検索
        if iyokan_results_count > 0:
            try:
                resp_iyokan = self.client.search(
                    query=query, search_depth="advanced", include_raw_content="markdown",
                    include_answer=False, include_domains=["iyokannet.jp"],
                    max_results=iyokan_results_count, chunks_per_source=3, timeout=120
                )
                _process_results(resp_iyokan.get("results", []), is_iyokan=True)
            except Exception as e:
```

```
                print(f"Error searching iyokannet.jp: {e}")

        # 2. ウェブ全体を検索（追加が有効な場合）
        if web_results_count > 0:
            try:
                resp_web = self.client.search(
                    query=query, search_depth="advanced", include_raw_content="markdown",
                    include_answer=False, max_results=web_results_count,
                    chunks_per_source=3, timeout=120
                )
                _process_results(resp_web.get("results", []), is_iyokan=False)
            except Exception as e:
                print(f"Error searching web: {e}")

        return items

    def _clean_text(self, text: str) -> str:
        if not text:
            return ""
        # Markdown/HTML → テキスト
        soup = BeautifulSoup(text, "html.parser")
        txt = soup.get_text(separator="n")
        txt = html.unescape(txt)
        txt = re.sub(r"n{3,}", "nn", txt)
        txt = re.sub(r"s+", " ", txt)
        # いよ観ネットの原文転載を避けるため、チャンク化前に短縮
        return txt.strip()
```

4. 埋め込みとベクトル検索による RAG

収集したテキストはそのまま生成器に渡さず，一度チャンクに分割して埋め込みベクトルへ変換する。本アプリでは `gemini-embedding-001` を使い，`FAISS` が利用可能であればそのインデックスを用い，利用できない環境では NumPy によるコサイン類似度計算へ切り替える。この実装により，環境差に強い RAG パイプラインを保ちながら，ユーザーの検索語に近い文脈だけを抽出できる。

コード 3: 埋め込みと検索基盤（`rag/retriever.py` より）

```
    # --- 2) 埋め込みユーティリティ ---
    def _embed(self, texts: List[str], task_type: str, dim: int = 768) -> np.ndarray:
        all_vecs = []
        # Process in batches to respect API limits (e.g., 100 texts per batch)
        # and add a delay to respect rate limits (e.g., 60 requests per minute)
        batch_size = 100

        for i in range(0, len(texts), batch_size):
            batch_texts = texts[i : i + batch_size]
            if not batch_texts:
                continue

            cfg = types.EmbedContentConfig(task_type=task_type, output_dimensionality=dim)
            try:
                res = self.gclient.models.embed_content(
                    model="gemini-embedding-001",
                    contents=batch_texts,
                    config=cfg,
                )

                batch_vecs = [np.array(e.values, dtype="float32") for e in res.embeddings]
                all_vecs.extend(batch_vecs)

                # If there are more batches to process, wait to avoid hitting rate limits.
                if i + batch_size < len(texts):
                    print(f"Embedding batch {i//batch_size + 1} complete. Waiting for 5 seconds...")
                    time.sleep(5)

            except Exception as e:
                print(f"An error occurred during embedding batch {i//batch_size + 1}: {e}")
                # Continue to the next batch if one fails
                continue

        if not all_vecs:
            # Handle case where all embedding calls failed
            return np.array([], dtype="float32").reshape(0, dim)

        return np.vstack(all_vecs)

    # --- 3) ベクトル化 → 検索 ---
    def _build_index(self, chunks: List[str]):
        X = self._embed(chunks, task_type="RETRIEVAL_DOCUMENT")
        if USE_FAISS:
            index = faiss.IndexFlatIP(X.shape[1])
            faiss.normalize_L2(X)
            index.add(X)
            return index, X
        # 代替: 生行列を返す
        return None, X

    def _search_index(self, index, X: np.ndarray, q: np.ndarray, topk: int = 8) -> Tuple[List[int], List[float]]:
        if USE_FAISS and index is not None:
            faiss.normalize_L2(q)
```

```
            D, I = index.search(q, topk)
            return I[0].tolist(), D[0].tolist()
        # 代替: NumPy コサイン類似度
        # 正規化
        Xn = X / (np.linalg.norm(X, axis=1, keepdims=True) + 1e-9)
        qn = q / (np.linalg.norm(q, axis=1, keepdims=True) + 1e-9)
        sims = Xn @ qn.T
        order = np.argsort(-sims[:, 0])[:topk]
        return order.tolist(), sims[order, 0].tolist()
```

さらに retrieve_for_plan では，単に類似度の高いチャンクを並べるのではなく，「1 URL あたり 1 チャンク」に絞ることで情報源の偏りを抑えている。また，選ばれたチャンク群は Gemini に一括で要約させる。このまとめ要約は API 呼び出し回数の削減につながるだけでなく，観光記事の原文を長く引用しないという方針にも合致している。

コード 4: チャンク選択と一括要約（rag/retriever.py より）

```
    def retrieve_for_plan(self, items: List[RetrievalItem], user_query: str, k: int = 8):
        chunk_texts, chunk_meta = [], []
        for it in items:
            chunks = self._chunk(it.content)
            for ch in chunks:
                chunk_texts.append(ch)
                chunk_meta.append({"title": it.title, "url": it.url, "site": it.site})

        if not chunk_texts:
            return [], []

        index, X = self._build_index(chunk_texts)
        if X.shape[0] == 0:
            return [], []

        q = self._embed([user_query], task_type="RETRIEVAL_QUERY")
        ids, scores = self._search_index(index, X, q, topk=k)

        # 1) まず「URL あたり 1 チャンク」に絞る（ここが効く）
        picked = []
        used_sources = []
        seen_urls = set()

        for idx in ids:
            meta = chunk_meta[idx]
            url = meta["url"]
            if url in seen_urls:
                continue
            seen_urls.add(url)
            used_sources.append(meta)
            picked.append(idx)
            # 無料枠前提なら 4〜5 程度が安全（任意で調整）
            if len(picked) >= min(k, 5):
                break

        if not picked:
            return [], []

        # 2) 要約をまとめて 1 回だけ実行
        texts_to_sum = [chunk_texts[idx] for idx in picked]
        print(f"Summarizing {len(texts_to_sum)} chunks in one request...")
        summaries = self._summarize_for_context_batch(texts_to_sum)

        selected = []
        for idx, summary in zip(picked, summaries):
            meta = chunk_meta[idx]
            selected.append(f"出典: {meta['title']} | {meta['url']}\n 要点:\n{summary}")

        return selected, used_sources


    def _summarize_for_context(self, text: str) -> str:
        # いよ観ネットのポリシーに配慮: 引用ではなく短い要点箇条書き
        prompt = (
            "以下の観光記事テキストから、固有名詞と実用情報（場所・体験・時期・所要時間・注意点）を日本語で 5 点以内に簡潔要約してください。n"
            "**原文の連続した引用は禁止。必ず言い換え・要約で**。最大 400 字。nn" + text[:4000]
        )
        resp = self.gclient.models.generate_content(
            model="gemini-2.5-flash-lite",
            contents=prompt,
            config=types.GenerateContentConfig(
                thinking_config=types.ThinkingConfig(thinking_budget=0),
            ),
        )
        return resp.text.strip()

    def _summarize_for_context_batch(self, texts: List[str]) -> List[str]:
        """
        複数チャンクを 1 回の generate_content で要約して返す（順序維持）。
        返り値は texts と同じ長さの summary リスト。
        """
        # JSON で返させる（順序通りの配列）
        joined = "\n\n".join(
            [f"### CHUNK {i+1}\n{t[:4000]}" for i, t in enumerate(texts)]
        )

        prompt = (
            "以下の複数の観光記事テキストを、それぞれ日本語で要点要約してください。\n"
            "- 各チャンクは固有名詞と実用情報（場所・体験・時期・所要時間・注意点）を中心に 5 点以内\n"
```

```
            "- 原文の連続した引用は禁止。必ず言い換え・要約で\n"
            "- 1 チャンクあたり最大 400 字\n"
            "出力は必ず JSON で、次の形式にしてください：\n"
            '{"summaries": ["要約 1", "要約 2", "..."]}\n\n'
            + joined
        )

        # 503/429 対策：軽いリトライ（必要最小限）
        last_err = None
        for attempt in range(4):
            try:
                resp = self.gclient.models.generate_content(
                    model="gemini-2.5-flash-lite",
                    contents=prompt,
                    config={
                        "response_mime_type": "application/json",
                        # thinking を切って軽量化したい場合（任意）
                        "thinking_config": {"thinking_budget": 0},
                    },
                )
                data = json.loads(resp.text)
                summaries = data.get("summaries", [])
                # 長さ不一致の保険
                if not isinstance(summaries, list) or len(summaries) != len(texts):
                    return ["" for _ in texts]
                return [str(s).strip() for s in summaries]
            except Exception as e:
                last_err = e
                msg = str(e)
                # 429/503 の場合だけ待って再試行
                if ("429" in msg) or ("RESOURCE_EXHAUSTED" in msg) or ("503" in msg) or ("UNAVAILABLE" in msg) or ("overloaded" in msg):
                    time.sleep(min(60, 2 ** attempt * 4))
                    continue
                raise
        raise last_err
```

5. プロンプト設計と構造化出力

検索で得られた文脈をもとに旅程を生成する段階では，自由文だけを返させるのではなく，JSON スキーマに従った構造化出力を要求している。プロンプト側では，長い原文引用の禁止，各日ごとの根拠 URL の提示，安全面や季節性への配慮などをガードレールとして明示する。また，発着地や宿泊地の扱いを具体的に指定することで，単なる観光地の列挙ではなく，日程として実行しやすいプランになるよう工夫している。

コード 5: 旅程生成プロンプト（rag/prompts.py より）

```
SYSTEM_GUARDRAILS = (
    "あなたは愛媛旅行の日本語プランナー。いよ観ネットの要約から**パラフレーズ**で情報を統合する。\n"
    "禁止: 原文の長い引用・転載、無根拠の情報。\n"
    "必須: 各日ごとに根拠 URL を列挙し、家族/高齢者/子連れなど安全面と季節性を明示。\n"
)

TEMPLATE = '''{system}
【旅行条件】
- 日数: {trip_days}日
- 開始日: {start_date}
- 同行者: {party}
- 移動手段: {transport}
- 関心テーマ: {interests}
- 主な訪問エリア: {start_area}
- 子連れ配慮: {with_kids}
- ペース: {pace}
- 発着地: {start_end_point}

【参考要点】
{context}

【指示】
上記条件に基づき、以下の点を厳守して現実的な旅行プランを JSON 形式で作成してください。

1.  **行程の起点と終点:** 1 日目は「{start_end_point}」から出発し、最終日（{trip_days}日目）は「{start_end_point}」に到着して解散する行程とします。
2.  **宿泊地の最適化:** **毎日、発着地に戻る必要はありません。** 各日の宿泊地は、その日の観光エリアや翌日の移動を考慮して、最も効率的で現実的な場所（例: 松山市内、道後温泉、今治市、宇和島市など）を設定してください。
3.  **時間配分:** 各アクティビティの所要時間と、エリア間の移動時間を考慮した、現実的な時間割を作成してください。
4.  **出力形式:** 必ず指定された JSON スキーマに従ってください。
'''

def build_plan_prompt(
    trip_days: int,
    start_date: str,
    party: str,
    transport: str,
    interests: list[str],
    start_area: str,
    with_kids: bool,
    pace: str,
    start_end_point: str,
    sources: list[dict],
```

```
    context: list[str],
) -> str:
    ctx = "\n\n".join(context)
    system = SYSTEM_GUARDRAILS

    start_end_prompt_val = start_end_point if start_end_point and start_end_point != "指定なし" else "指定なし"

    return TEMPLATE.format(
        system=system,
        trip_days=trip_days,
        start_date=start_date,
        party=party,
        transport=transport,
        interests=",".join(interests),
        start_area=start_area,
        with_kids=with_kids,
        pace=pace,
        start_end_point=start_end_prompt_val,
        context=ctx,
    )
```

実際の生成処理では，上記プロンプトと抽出済みコンテキストを `generate_content` に渡し，`response_json_schema` を指定して旅程の構造を固定する。これにより，生成結果をそのまま画面表示や Markdown 出力へつなげやすくなる。

コード 6: 初回プラン生成（`app.py` より）

```
    with st.spinner("RAG で関連チャンクを選別中..."):
        top_chunks, used_sources = retriever.retrieve_for_plan(
            items=[RetrievalItem(**i) for i in items_state],
            user_query=query,
            k=8,
        )

    with st.spinner("Gemini で旅程を構成中..."):
        prompt = build_plan_prompt(
            trip_days=trip_days, start_date=str(start_date), party=party,
            transport=transport, interests=interests, start_area=start_area,
            with_kids=with_kids, pace=pace, start_end_point=start_end_point,
            sources=used_sources, context=top_chunks,
        )
        resp = client.models.generate_content(
            model="gemini-2.5-flash-lite",
            contents=prompt,
            config={
            "response_mime_type": "application/json",
            "response_json_schema": ITINERARY_SCHEMA,  # JSON Schema (dict)
            },
        )

        st.session_state.plan_json = json.loads(resp.text)
        st.session_state.messages = [{
            "role": "assistant",
            "content": "プランの初稿を作成しました。変更したい点があれば、下のチャット欄から具体的に教えてください。\n（例: 2日目はもっとゆったりしたプランにして、〇〇を追加して）"
        }]
        st.rerun()
```

6. チャットによる再計画

本アプリでは，初回生成のあとにチャットで修正依頼を受け取り，既存プランを保ちながら再構成する流れを用意している。利用者は「2 日目をもっとゆったりにしたい」「温泉を追加したい」といった自然文で修正を依頼できる。

コード 7: チャットによるプラン修正（`app.py` より）

```
if prompt := st.chat_input("プランの修正点を入力してください"):
    if not st.session_state.plan_json:
        st.warning("まずプランを生成してください。")
        st.stop()

    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    with st.chat_message("assistant"):
        with st.spinner("プランを修正中..."):
            refine_prompt = build_refine_plan_prompt(
                existing_plan=st.session_state.plan_json,
                user_request=prompt,
```

```
        )
        resp = client.models.generate_content(
            model="gemini-2.5-flash-lite",
            contents=refine_prompt,
            config={
                "response_mime_type": "application/json",
                "response_json_schema": ITINERARY_SCHEMA,
            },
        )

        try:
            new_plan = json.loads(resp.text)
            st.session_state.plan_json = new_plan
            response_text = "プランを修正しました。いかがでしょうか？ さらに修正したい点があれば、教えてください。"
        except json.JSONDecodeError:
            response_text = "プランの修正に失敗しました。形式が正しくないようです。もう一度試しますか？\n" + resp.text

        st.session_state.messages.append({"role": "assistant", "content": response_text})
        st.rerun()
```

(3) わかったこと・わからなかったこと

1. わかったこと

実際にシステムを動かしてみて，次の点が重要だと分かった。

- 検索対象を `iyokannet.jp` 中心に絞ることで，観光プランの根拠を明確にしやすい。
- 生成前に関連チャンクを選別し，要点だけをまとめて渡すことで，長い観光記事をそのまま扱うよりも安定した旅程が得られる。
- JSON スキーマによる構造化出力は，表示，保存，再編集のすべてに有効であり，単なるテキスト生成より実用性が高い。
- チャットで再計画できる構成にしておくと，初回出力をたたき台として徐々に利用者の意図へ寄せられる。

2. わからなかったこと・この講義で解消したいこと

一方で，実装してみてまだ十分に整理できていない点も残った。

- 埋め込みベクトルを用いると，なぜ意味的に近い観光文書どうしが近い位置に配置され，検索に有効になるのかをより深く理解したい。表現学習や自己教師あり学習の観点から仕組みを整理したい。
- 今回は Gemini を用いて旅行プランを生成したが，Transformer や大規模言語モデルが，従来の系列モデルよりどのように文脈を扱い，長い入力から要点を取り出しているのかを講義で学びたい。
- RAG を使っても，生成結果が不正確になったり，根拠の弱い内容を含んだりすることがある。深層生成モデルや自然言語処理の観点から，そのような誤りがなぜ起こるのか，またどうすれば抑えられるのかを理解したい。

参考リンク

- Ehime Tour Planner アプリ公開リンク:
  `https://ehime-tour-planner-iicqpkkbfs9zrcjba6a9at.streamlit.app/`